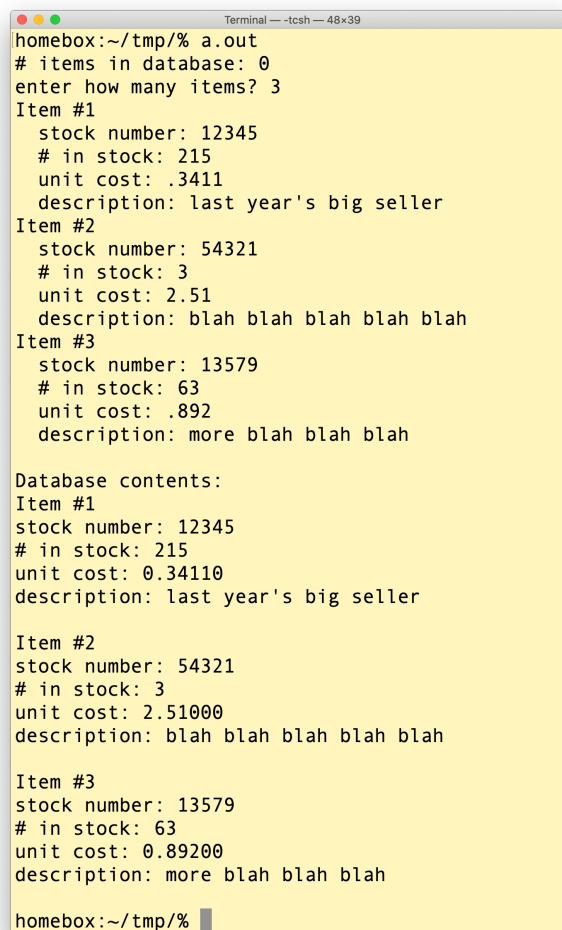


Lab #3 EGRE246

For this lab you are going to write a few functions to input and output structures. Note that the actual amount of code you will write is fairly small and straight-forward; a lot of your time in this lab will be spent understanding the code I give you and figuring out what I am asking you to do.

I have given you the shell of a program (on Canvas) for a very simple widget “database” – really just a list of structures. Each item in this database has 4 fields representing a widget id number, the number of items in stock, the wholesale cost per item, and a brief description of the item. Your finished program will input several items (widgets) for your database and then print these items out. Here is a sample run of the code I give you; note that I input the number of widgets first, followed by typing in the data for these widgets, and finally I output all of the data stored in the database:



```
homebox:~/tmp/% a.out
# items in database: 0
enter how many items? 3
Item #1
  stock number: 12345
  # in stock: 215
  unit cost: .3411
  description: last year's big seller
Item #2
  stock number: 54321
  # in stock: 3
  unit cost: 2.51
  description: blah blah blah blah blah
Item #3
  stock number: 13579
  # in stock: 63
  unit cost: .892
  description: more blah blah blah

Database contents:
Item #1
stock number: 12345
# in stock: 215
unit cost: 0.34110
description: last year's big seller

Item #2
stock number: 54321
# in stock: 3
unit cost: 2.51000
description: blah blah blah blah blah

Item #3
stock number: 13579
# in stock: 63
unit cost: 0.89200
description: more blah blah blah

homebox:~/tmp/%
```

Here is the program available for downloading on Canvas for this lab:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h> // for strcpy
#include <ctype.h>  // for isspace

#define TEXT_LENGTH 250
#define MAX_DB_SIZE 1000
#define MAX_DESCRIPTION_SIZE 160

typedef struct wType {
    int id; // widget id number
    int num; // number in stock
    float cost; // our unit cost
    char *des; // description of item
} widget;

void initDB(widget *[],int); // initialize database (db) to all NULL
int count(widget *[],int); // returns count of actual items in db
widget *newItem(int, int, float, char *); // creates and returns a new db item
void printItem(widget *); // prints a single db item
void printDB(widget*[],int); // prints all db items

////////////////////////////////////

int main(void){
    widget *db[MAX_DB_SIZE];
    int total;

    initDB(db,MAX_DB_SIZE);
    printf("# items in database: %d\n",count(db,MAX_DB_SIZE));
    printf("enter how many items? ");
    scanf("%d",&total);
    if(total <= 0) exit(EXIT_SUCCESS);

    int id,num; float cost; char des[MAX_DESCRIPTION_SIZE], ch;
    des[0] = '\0';
    for(int i = 0; i < total; i++) {
        printf("Item #%d\n",i+1);
        printf("  stock number: "); scanf("%d",&id);
        printf("  # in stock: ");   scanf("%d",&num);
        printf("  unit cost: ");    scanf("%f",&cost);
        printf("  description: ");
        while(isspace(ch=getchar())) ; // see notes in handout on this
        ungetc(ch,stdin);
        scanf("%[^\n]",des);
        db[i] = newItem(id,num,cost,des);
    }
    printf("\nDatabase contents:\n");
    printDB(db,total);
    return 0;
}

```

Complete the following steps for this lab.

1. Download and save the file listed above off of the Canvas class web pages. This program will not compile as is; you will need to write the 5 functions declared in the prototypes. I will give you most of this code.
2. Comment out code until the program compiles with no syntax errors.
3. Begin by writing the function `newItem` and `printItem`; most of the code is listed below. When finished with these 2 routines write calls to them to test if they are working correctly.

```
widget *newItem(int new_id, int new_num, float new_cost, char *new_des) {
    widget *tmp = malloc(sizeof(widget)); // <-- make sure you understand this!
    tmp->id = new_id;
    // you fill in more code here!
    tmp->des = malloc(strlen(new_des)+1); // <-- make sure you understand this!
    strcpy(tmp->des,new_des);
    return tmp;
}

void printItem(widget *item) {
    printf("stock number: %d\n", item->id);
    // more code here - remember there are 4 fields
}
```

4. Write the `initDB` function. Our database variable (`db`) is just a list of pointers of size `MAX_DB_SIZE`; how do we specify that a cell is empty? Usually for pointer variables we indicate it is empty by storing `NULL` in each cell.

```
void initDB(widget *list[],int n) {
    if (n < 1) return;
    for(int i=0; i < n; i++)
        // your code here - one line!
}
```

Make sure to write driver code to test your routine before you move on.

5. One of the easier routines is `count`. All this routine does is count and return the number of cells in the database that are not equal to `NULL`.

```
int count(widget *list[], int n) {
    if (n <= 0) return 0;
    int c = 0;
    for(int i = 0; i < n; i++)
        // your code here
}
```

Again, test your code before moving on!

6. The final function is `printDB`.

```

void printDB(widget *db[],int size) {
    int c = ... // c should store the number of actual widgets in the db
    if(c <= 0) { printf("<database empty>\n"); return; }
    for(int i = 0; i < c; i++) {
        // now use a call to printItem to print out each item
    }
}

```

Now if everything has been coded up correctly the sample program should run!

FINISHED!

Inputting a Whole Line of Text in C

How do you input a whole line of text into a string variable (say `str`) in C? You might be tempted to try this:

```
scanf("%s",str);
```

However, remember that this only reads until it hits a white space character! What do you do if you wish to input *all* characters up until you reach a newline?

If you search online for solutions you might be tempted to use `gets`, `fgets`, or the non-standard `getline`. And whereas these would work, they all have a flaw (in my opinion) that is annoying – they all put the newline character in the string!

Also it's nice to use `scanf` for all your input if you are using it for some of it (as we are). Why? Because `scanf` will consistently skip leading white space and multiple calls to `scanf` then just work.

My favorite solution: use regular expressions with `scanf`, something we have not covered and won't actually do so now. Try this:

```
scanf("%[^\n]",str);
```

That `%[^\n]` defines the regular expression character class that `scanf` will use to build its string. It means “insert all characters up to but not including a newline into a string” (the `[ ]` delimits a character class, and the `^\n` means everything but a newline). And this works wonderfully ... almost.

The problem arise if you do something like this:

```

int n;
scanf("%d",&n);      // scanf #1
scanf("%[^\n]",str); // scanf #2

```

If you input `5 hi mom`[return] (case #1) then `n` would be set to 5 and `str` to the string “`hi mom`”. Or if instead your tried

```
5[return]
hi mom[return]
```

across two lines (case #2) `n` would again be 5 but `str` would be empty (i.e. length 0)! **Why?**

First for case #1: `scanf #1` will leave a white space in the input buffer. So that space will be included in `str`.

In case #2, `scanf #1` will leave a newline in the buffer. And since the first character in the buffer is now a newline, and `scanf #2` reads everything up to but not including a newline, there's nothing for it to read so `str` becomes an empty string!

The very-C-solution is to always flush the `stdin` (keyboard) buffer of all white space before calling a `scanf` like #2. Like this:

```
while((ch = getchar()) == '\n') ;
ungetc(ch,stdin);
```

The empty-body-loop just reads in and throws away all characters that are newlines. However, the last character it throws away *needs to be the first character in `str`*! So we use `ungetc` to push it back into the `stdin` keyboard buffer before calling `scanf #2`. And it works.

This code to input an entire line of text up to but not including a newline is very handy to note and remember as it's something you often need to do in programs. I store the code to do it in a special test program so I always have it available to cut-and-paste into other code should I need it.